

Tarea no. 2 . JS



- Docente: Mario Humberto Rodríguez Chávez
- Carrera: Ing. Tecnologías de la Información e Innovación Digital
- Alumno: Alexa Mendoza Hernández-2530305
- Materia: Aplicaciones Web

Introducción y contexto

1. ¿Qué es JavaScript y cuáles son sus principales usos en la web?

JavaScript es un lenguaje de programación interpretado, esto quiere decir que necesita de un intérprete para su ejecución. En la mayoría de los casos es interpretado gracias a los navegadores, como chrome, edge, Firefox.

JavaScript sirve para crear de todo un poco:

- **Páginas web (lo que ves):** Diseña la parte visual, haciendo que los sitios sean interactivos y animados.
- **Servidores (lo que no ves):** Controla la lógica interna y las bases de datos de una web.
- **Aplicaciones móviles:** Permite crear apps para celulares.
- **Videojuegos:** Permite hacer juegos sencillos que funcionen directo en el navegador de internet, sin instalar nada.

2. ¿En qué año y con qué propósito se creó JavaScript?

JavaScript fue creado en 1995 por Brendan Eich, su objetivo era permitir interactividad en las páginas web, algo revolucionario para la época. En solo 10 días, Eich desarrolló la primera versión de **JavaScript**, que se integró en el navegador Netscape Navigator.

3. ¿Por qué JavaScript no es lo mismo que Java?

Java es pesado y estructurado, usado para construir los motores de aplicaciones grandes como las de un banco o Android. **JavaScript** es ligero y ágil, pensado para ponerle botones interactivos, animaciones y color a los sitios web.

4. ¿Cómo se complementa JavaScript con HTML y CSS?

Los tres lenguajes son parte integrante de la creación de una página web funcional y visualmente atractiva. **HTML** proporciona el contenido y la estructura, **CSS** añade el diseño y el estilo, y **JavaScript** aporta elementos interactivos. Trabajan juntos para dar vida a un sitio web.

Fundamentos del lenguaje

5. ¿Cuál es la sintaxis básica de JavaScript y cómo se escriben comentarios?

Sintaxis:

→ Variables (para guardar información) :

Hoy en día se utilizan principalmente `let` (para valores que cambian) y `const` (para valores fijos):

```
const nombre = "Ana"; // No cambia
```

```
let edad = 25; // Puede cambiar más adelante
```

→ Estructuras de control (para tomar decisiones o repetir acciones) :

Se usan condiciones (`if`) y bucles (`for`, `while`):

```
if (edad >= 18) {  
    console.log("Es mayor de edad");  
} else {  
    console.log("Es menor de edad");  
}
```

→ Funciones (para reutilizar código):

Bloques de código que realizan una tarea específica:

```
function saludar(persona) {  
    return "¡Hola, " + persona + "!";  
}  
// Para usarla:  
saludar("Ana");
```

→ Cómo se escriben comentarios:

De una sola línea: Se usa dos barras diagonales

```
// al inicio.
```

De varias líneas: Inicia con `/*` y termina con `*/`.

6. ¿Qué diferencias hay entre var, let y const?

- **Var y let:** Permiten cambiar su valor en cualquier momento.
- **Const:** : Es una constante. Una vez que le asignas un valor, no puedes volver a asignarle uno nuevo (dará un error).

7. ¿Cuáles son los tipos de datos primitivos en JavaScript? Da un ejemplo de cada uno.

→ String (texto):

Representa cadenas de texto. Se escribe entre comillas simples, dobles o invertidas:

```
const nombre = "Carlos";
```

→ Number (Número) :

Representa valores numéricos, tanto enteros como con decimales.

```
const edad = 30;  
const precio = 19.99;
```

→ Boolean (Booleano) :

Solo puede tener dos valores: **true** (verdadero) o **false** (falso). Se usa para tomar decisiones.

```
const esMayorDeEdad = true;
```

→ Undefined (Indefinido) :

Es el valor que tiene automáticamente una variable que ha sido declarada pero aún no se le ha asignado ningún valor.

```
let valor;  
console.log(valor); // Imprime: undefined
```

→ Null (Nulo) :

Representa la ausencia intencional de un valor o un valor "vacío". A diferencia de **undefined**, este lo asignas tú a propósito:

```
const respuestaServidor = null;
```

→ **Symbol (Símbolo) :**

Representa un identificador único e irrepetible. Se utiliza principalmente para crear claves de propiedades privadas o únicas en objetos.

`const idUnico = Symbol("id");`

8. ¿Qué diferencia hay entre un array y un objeto en JavaScript?

- Un **Array** es como una **lista del supermercado**: importa el orden en el que pones los elementos y los buscas contando las posiciones (*el primero, el segundo, el tercero...*). Usa corchetes
- Un **Objeto** es como una **ficha de información o ID**: guarda datos etiquetados con un nombre específico . No importa el orden, buscas los datos usando su etiqueta. Usa llaves

9. ¿Qué son y cómo se usan los operadores aritméticos, lógicos y relacionales?

→ **Operadores Aritméticos:**

Sirven para realizar cálculos matemáticos básicos y avanzados.

Operadores	Nombre	Ejemplo	Resultado
+	Suma	5+3	8
-	Resta	10-4	6
*	Multiplicación	4*2	8
/	División	15/3	3
%	Módulo (residuo de una división)	7%3	1
**	Exponenciación (potencia)	2**3	8

→ Operadores Relacionales (o de Comparación)

Sirven para comparar dos valores. Siempre devuelven un valor booleano:

`true` (verdadero) o `false` (falso).

Operadores	Significado	Ejemplo	Resultado
<code>></code>	Mayor que	<code>10 > 5</code>	<code>true</code>
<code><</code>	Menor que	<code>3 < 1</code>	<code>false</code>
<code>>=</code>	Mayor o igual que	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Menor o igual que	<code>4 <= 2</code>	<code>false</code>
<code>===</code>	Igualdad estricta (mismo valor y tipo)	<code>5 === "5"</code>	<code>false</code>
<code>!==</code>	Desigualdad estricta (diferente valor o tipo)	<code>5 !== 3</code>	<code>true</code>

→ Operadores Lógicos

Sirven para combinar varias condiciones. Operan con valores booleanos y devuelven `true` o `false`.

Operadores	Nombre	Descripción
<code>&&</code>	AND (Y)	Devuelve <code>true</code> solo si ambas condiciones son verdaderas.
<code> </code>	OR (O)	Devuelve <code>true</code> si al menos una condición es verdadera.
<code>!</code>	NOT (NO)	Invierte el valor (si era <code>true</code> pasa a <code>false</code> y viceversa).

Control de flujo

10. ¿Cómo funcionan las sentencias condicionales **if**, **else** y **switch**?

- **if**: Evalúa una primera condición. Si es verdadera, ejecuta su bloque de código.
- **else if**: Si la condición anterior fue falsa, evalúa una nueva condición opcional.
- **else**: Sirve como caso por defecto; se ejecuta únicamente si ninguna de las condiciones anteriores fue verdadera.

11. ¿Cuáles son las diferencias entre los bucles **for**, **while** y **do...while**?

- **Bucle for**: Se utiliza cuando sabes exactamente cuántas veces quieres repetir la acción desde el principio.

Agrupa en una sola línea tres partes clave: la inicialización de la variable, la condición de parada y el incremento/decremento.
- **Bucle while** : Se utiliza cuando NO sabes cuántas veces se va a repetir la acción, sino que depende de una condición dinámica (por ejemplo, esperar a que el usuario presione un botón o ingrese una clave válida).

Evalúa la condición ANTES de ejecutar el código. Si la condición es falsa desde el inicio, el código de adentro nunca se ejecuta.

- **Bucle do while**: Es similar al while, pero con una diferencia crítica: evalúa la condición DESPUÉS de ejecutar el código.

Esto garantiza que el bloque de código se ejecutará al menos una vez, sin importar si la condición es falsa desde el principio.

12. ¿Para qué sirven las palabras clave **break** y **continue**?

Ambas te dan el control dentro de un bucle: **break** funciona como un freno de emergencia que cancela el ciclo por completo, mientras que **continue** solo se salta el paso actual y pasa directo al siguiente sin detener el proceso. Son las herramientas para decidir cuándo frenar la repetición o ignorar ciertos datos.

Funciones

13. ¿Cómo se declara una función y cómo se llama en JavaScript?

→ (Función nombrada) :

Es la forma más clásica. Usa la palabra reservada **function** seguida del nombre.

```
function saludar(nombre) {  
    return "¡Hola, " + nombre + "!";  
}
```

14. ¿Qué significa que una función tenga parámetros y un valor de retorno?

Los parámetros son los datos que le entran a la función para que pueda trabajar. El valor de retorno es la respuesta o resultado final que la función te devuelve al terminar. Sin parámetros no tiene con qué calcular, y sin retorno no te entrega nada.

15. ¿Qué son las funciones flecha (=>) y qué ventajas tienen?

Son una sintaxis más corta y moderna para crear funciones en JavaScript. Su ventaja principal es que reducen el código al no requerir la palabra **function** ni el **return** en líneas simples. Además, si reciben un solo parámetro puedes omitir los paréntesis.

16. ¿Qué es el scope en JavaScript y cómo afecta a las variables?

El scope (o alcance) es la zona del código donde una variable existe y se puede usar. Si se intenta usar una variable fuera de su alcance, JavaScript dará un error porque simplemente no la encuentra.

¿Cómo afecta?

El tipo de palabra clave que usas determina qué tanto se limita tu variable:

- **let y const:** Tienen **scope de bloque**. Se quedan encerradas dentro del `{ }` o de la función donde las creaste. Son las más seguras.
- **var:** Ignora los bloques `{ }` (como los de un `if` o un bucle) y solo respeta el scope de función. Por eso puede causar errores accidentales y ya casi no se usa.

Objetos y colecciones

17. ¿Cómo se declara un objeto y cómo se accede a sus propiedades?

Un objeto es como una ficha de datos guardada entre llaves {} con parejas de nombre: valor.

```
// 1. Se declara con llaves
const usuario = { nombre: "Ana", edad: 25 };
// 2. Se leen sus datos con un punto
console.log(usuario.nombre); // Imprime: Ana
// 3. Se usan corchetes si el dato viene de una variable
console.log(usuario['edad']); // Imprime: 25
```

18. ¿Qué diferencia hay entre una propiedad y un método en un objeto?

- **Propiedad:** Es una variable guardada en el objeto. Contiene un dato o valor (un texto, un número, un booleano). Es una característica.
- **Método:** Es una función guardada en el objeto. Es una acción que el objeto puede realizar.

19. Menciona al menos 3 métodos básicos de los arrays (push, pop, map, etc.) y explica cómo funcionan.

Los métodos de un array son como herramientas para manipular tu lista de datos:

- **push():** Agrega un elemento al final de la lista.
- **pop():** Saca y elimina el último elemento de la lista.
- **map():** Transforma toda la lista creando una nueva con cambios aplicados a cada elemento.

Supongamos el ejemplo de alimentos: Con **push** añadimos un alimento al carrito de compras, con **pop** quitamos el último que metimos, y con **map** le aplicamos un descuento a todos los precios de la lista.

Otros temas básicos

20. ¿Qué es el DOM y por qué es importante en el uso de JavaScript en páginas web?

El DOM es como un mapa que convierte toda la página web en elementos que JavaScript puede entender y controlar.

Su importancia radica en que le da superpoderes a la página: permite modificar textos, cambiar colores, esconder cosas o responder al instante cuando el usuario hace clic, todo en vivo y sin recargar la web.

21. ¿Qué es un evento en JavaScript y menciona al menos 3 ejemplos comunes?

Un evento es cualquier acción o suceso que pasa en la página web y que JavaScript puede detectar para dar una respuesta.

3 ejemplos comunes:

- **click**: Ocurre cuando el usuario presiona un botón o elemento.
- **submit**: Se activa cuando la persona envía los datos de un formulario.
- **keydown**: Sucede en el instante en que alguien presiona una tecla en el teclado.

22. ¿Cuál es la diferencia entre alert, prompt y console.log?

Las tres sirven para mostrar o pedir información, pero cambian en dónde y a quién se la enseñan:

- **alert()**: Muestra una ventana emergente en la pantalla para avisarle algo al usuario (detiene la página hasta que da clic en "Aceptar").
- **prompt()**: Muestra una ventana emergente para pedirle al usuario que escriba un dato.
- **console.log()**: Muestra un mensaje oculto en las herramientas de desarrollador para que uno como programador revise su código (el usuario normal no lo ve).

23. ¿Cómo se abre la consola del navegador y qué ventajas tiene usarla para depurar código?

La consola del navegador es una pantalla oculta donde puedes probar código y ver los errores de tu página al instante.

- **¿Cómo abrirla?:** Se presiona la tecla F12 (o se le da clic derecho en cualquier parte de la página y se elige Inspeccionar), y luego se le da clic en la pestaña que dice Consola.

Ventajas para depurar:

- **Te avisa de los errores:** Si algo falla en tu código, la consola te muestra el problema en texto rojo y te dice exactamente en qué línea ocurrió.
- **Pruebas al instante:** Puedes escribir y probar líneas de código directamente en ella para ver qué pasa sin tener que guardar o reiniciar nada.